阿里云天池 Python 训练营 - Part3

18. Python 自定义函数

函数是Python编程中的核心内容之一。

函数的最大优点是可以增强代码的重用性和可读性。

Python 中有很多内置函数，但内置函数并不能满足用户的需求，这时用户可以创建自定义函数。

本章将介绍如何自定义函数。

18.1 自定义函数编写规范

Python中的自定义函数需要遵循一定的规范，比如定义时的关键字是什么、返回值关键字是什么、参数的写法，以及如何调用自定义函数等。下面来介绍一下相关的知识点。

18.1.1 函数的定义

在自定义函数时，有固定的语法格式。语法格式如

下：

```
def 函数名(参数)：
    函数体
    return 返回值
```

- def：函数的关键字，不能没有，也不能改，有此关键字就表示是自定义函数。
    - 函数名：函数的名称，根据函数名调用函数。
    - 函数名是包含字母、数字、下划线的任意组合，不能以数字开头。
    - 函数名可以任意命名，建议尽量定义成可以表示函数功能的名称。
- 参数：为函数体提供数据。
    - 参数是被放在小括号中的，
    - 可以设置多个参数，
    - 参数之间用逗号分隔。
    - 在小括号的后面加冒号。
- return 返回值：
    - 整体作为自定义函数的可选参数，用于设置自定义函数的返回值。
    - 也就是说，自定义函数可能有返回值，也可能没有返回值，视情况而定。

> 注意，函数体和返回值语句要做缩进处理，也就是从冒号开始换行的语句都要做缩进处理。

18.1.2 自定义函数的创建与调用

了解了自定义函数的语法格式后，下面学习自定义函数的创建与调用，比如创建一个单价与数量相乘的函数并调用。

> 因为代码是自上而下运行的，所以一定要先定义函数，再调用函数。

自定义函数案例的代码如下所示。

```
# 有返回值的自定义函数
def total_sum1(price,amount):
    money=price*amount
    return money

# 无返回值的自定义函数
def total_sum2(price,amount):
    str='单价:{} 数量:{} 金额:
{}'.format(price,amount,price*amount)
    print(str)

# 调用自定义函数
print(total_sum1(10,20))
total_sum2(10,20)
```

18.1.3 自定义函数应用案例：平均函数的定义及应用

及应用

本案例将每个人1～12月的工资进行平均，然后写入N列对应的单元格，如图所示。



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	姓名	1月	2月	3月	4月	5月	6月	7月	8月	9月	10月	11月	12月	平均工资
2	常威	6112元	9532元	10385元	8669元	10732元	6978元	11123元	7298元	8291元	5962元	11452元	11443元	
3	雷豹	7856元	10198元	11938元	11840元	8773元	10434元	6417元	11691元	9480元	8695元	9628元	10280元	
4	包不同	6369元	11078元	5787元	7541元	8426元	10079元	8800元	5822元	8702元	11098元	10865元	10747元	
5	宋福	10794元	5871元	9159元	7484元	7097元	10307元	9351元	6708元	5614元	5941元	11989元	11693元	

工资表

处理前

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	姓名	1月	2月	3月	4月	5月	6月	7月	8月	9月	10月	11月	12月	平均工资
2	常威	6112元	9532元	10385元	8669元	10732元	6978元	11123元	7298元	8291元	5962元	11452元	11443元	8998.08元
3	雷豹	7856元	10198元	11938元	11840元	8773元	10434元	6417元	11691元	9480元	8695元	9628元	10280元	9769.17元
4	包不同	6369元	11078元	5787元	7541元	8426元	10079元	8800元	5822元	8702元	11098元	10865元	10747元	8776.17元
5	宋福	10794元	5871元	9159元	7484元	7097元	10307元	9351元	6708元	5614元	5941元	11989元	11693元	8500.67元

工资表

处理后

本案例的编程思路是将每个人所有的工资求和，再除以月份数，按此思路自定义一个平均函数即可。

```
# 自定义平均函数
def average(lst):
    num=sum(lst)/len(lst) # 平均处理。
    avg=float('{:.2f}'.format(num)) # 格式化平均值。
    return avg # 返回平均值。

# 数据平均处理
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb=xlrd.open_workbook('./28-functions-1.xls');ws=wb.sheet_by_name('工资表') # 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('工资表') # 复制工作簿，以及读取副本工作簿下的工作表。
```

```
for row_num in range(1,ws.nrows): # 循环行号。
    salary_list=ws.row_values(row_num)[1:-1] # 获取1到12月份的工资列表。

nws.write(row_num,13,average(salary_list)) # 调用平均函数average对salary_list列表求平均。
nwb.save('/28-functions-2.xls') # 保存副本工作簿。
```

- 第 2～5 行代码自定义平均函数average(lst)。其中，lst 为必选参数，指定要进行平均计算的列表或元组。

- 第 13 行代码 nws.write(row*num,13,average(salary*list))，其中 average(salary_list) 表示使用 average 函数对每行的工资数据进行平均，然后将结果写入 N 列对应的单元格。

18.2 必选参数的写法及应用

必选参数就是函数调用时必须要传入值的参数，既不能多，也不能少。必选参数也可以叫作位置参数。

18.2.1 必选参数（位置参数）

下面是一个小案例，根据固定的分数确定等级，代码如下所示。

```
# 自定义函数。
def level(number,lv1,lv2,lv3):
    if number>=90:
        return lv1
    elif number>=60:
        return lv2
    elif number>=0:
        return lv3
# 自定义函数的调用。
for score in [95,63,58,69,41,88,96]:

print(score,level(score,'优','中','差'))
```

- 第 2～8 行代码自定义等级函数 level(number,lv1,lv2,lv3)，该函数的参数说明如下。
    - number:必选参数，输入用于判断的数字。
    - lv1:必选参数，当大于或等于 90 分时对应的等级。
    - lv2:必选参数，当大于或等于 60 分且小于 90 分时对应的等级。
    - lv3:必选参数，当大于或等于 0 分且小于 60 分时对应的等级。
- 第 10 行和第 11 行代码将 [95,63,58,69,41,88,96] 列表中的元素循环赋值给 score 变量，score 变量作为level 函数的第 1 个参数，将第 2、3、4

  个参数分别写入“优”“中”“差” 3 个等级。最后运行

print(score,level(score,'优','中','差')) 后，返回的结果如图所示。

```
95 优
63 中
58 差
69 中
41 差
88 中
96 优

Process finished with exit code 0
```

18.2.2 必选参数应用案例：给号码分段

本案例给“卡号表”工作表 B 列的卡号做分段处理，每 4 位分一段，如果不够 4 位，也要分成一段，如图所示。

处理前

	A	B	C
1	姓名	卡号	卡号分段
2	常威	595492888094924	
3	雷豹	464679984106816	
4	小林	755402768711247	
5	来福	548413737257575	
6	小周	607968439327173	
7	小曾	114768534364829	
8	周州	119663999124026	
9	黄风	367008500245562	
10	小梁	859462232201421	

卡号表

处理后

	A	B	C
1	姓名	卡号	卡号分段
2	常威	595492888094924	5954-9288-8094-924
3	雷豹	464679984106816	4646-7998-4106-816
4	小林	755402768711247	7554-0276-8711-247
5	来福	548413737257575	5484-1373-7257-575
6	小周	607968439327173	6079-6843-9327-173
7	小曾	114768534364829	1147-6853-4364-829
8	周州	119663999124026	1196-6399-9124-026
9	黄风	367008500245562	3670-0850-0245-562
10	小梁	859462232201421	8594-6223-2201-421

卡号表

本案例的编程思路是定义一个分段截取函数，可以让用户指定每段长度，也可以指定分段之间的分隔符，然后将B列中的卡号放到自定义函数中运行，最后将处理好的结果写入C列单元格。

```
# 自定义函数
def intercept(s,num,delimiter):
    s1 = str(s) # 将要分段的对象转换为字符串类型。
    lst = [s1[n:n+num] for n in range(0,len(s1),num)] # 对数据进行分段处理。
    s2 = delimiter.join(lst) # 合并分段的列表。
    return s2 # 将合并结果返回函数。

# 自定义函数应用
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb = xlrd.open_workbook(./29-functions-1.xls');ws=wb.sheet_by_name('卡号表') #
```

```
读取工作簿和工作表。
nwb = copy(wb);nws=nwb.get_sheet('卡号表') # 复制工作簿，以及读取副本工作簿下的工作表。
for row_num in range(1,ws.nrows): # 循环行号。
    val = ws.cell_value(row_num,1) # 读取B列单元格值。

nws.write(row_num,2,intercept(val,4,'-')) # 截取指定长度的字符进行分段，再写入单元格。
nwb.save('./29-functions-2.xls') # 保存副本工作簿。
```

- 第 2～6 行代码自定义切片函数 intercept(s,num,delimiter)，该函数的参数说明如下。
    - s: 必选参数，指定要分的段数。
    - num: 必选参数，表示每段的长度。
    - delimiter: 必选参数，表示每段之间的分隔符。
- 第 13 行代码 nws.write(row_num,2,intercept(val,4,'-'))，其中 intercept(val,4,'-'）部分使用自定义的intercept 函数，看一下此函数的实参，val 表示 B 列的每个卡号，每 4 个字符为一段，将"-"作为分隔符进行

  分段，最后将处理好的结果写入 C 列对应的单元

格。

18.3 可选参数的写法及应用

可选参数表示在调用函数时可传可不传该参数的值。

- 在定义可选参数时，要设置一个值，这个值叫作**默认值**，所以可选参数也叫作**默认参数**。
- 在调用函数时，如果可选参数**没有传入值**，则使用可选参数的**默认值**。

在自定义函数时，既有必选参数，又有可选参数，要先定义必选参数，再定义可选参数。在调用函数时也一样，要先传必选参数，再传可选参数。

18.3.1 可选参数（默认参数）

比如，现在要自定义一个类似于 Excel 中的 MID 函数的函数，在对序列对象做切片时，可以指定位置和截取长度。案例代码如下所示。

```
# 自定义函数。
def mid(iterable,start = 0,length = 1):
    return iterable[start:start+length]
# 自定义函数调用。
print(mid('abcdefgh'))
print(mid('abcdefgh',2))
print(mid('abcdefgh',2,4))
```

- 第 2～3 行代码自定义函数 mid(iterable，start＝0，length＝1)，该函数的参数说明如下

0，length－1)，该函数的参数说明如下。

  - iterable:必选参数，提供可以被切片的对象，比如字符串、列表、元组。
  - start:可选参数，切片的起始位置，默认值是0，也就是从开始的位置切片。
  - length:可选参数，切片的长度，默认值是1。

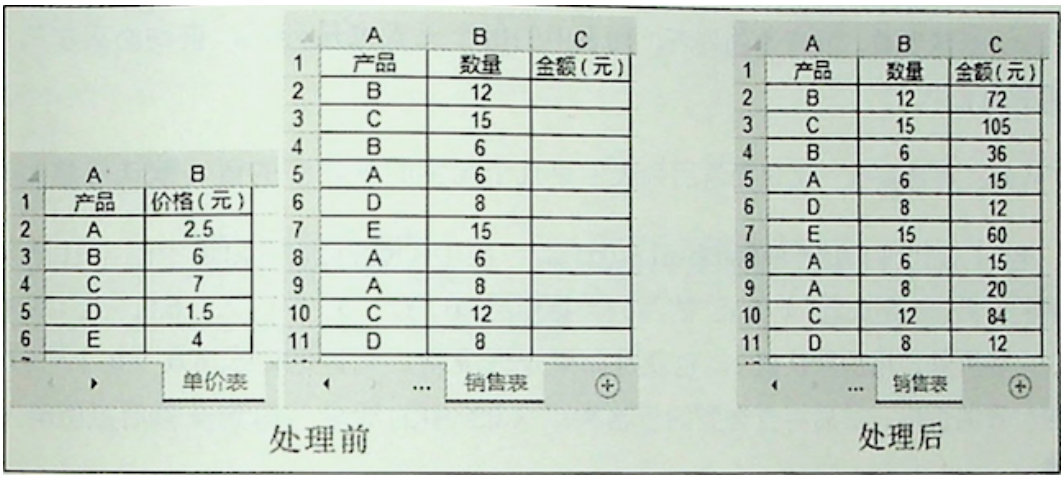
- 第 5～7 行代码调用mid函数做测试。

  - 第5行代码 print(mid('abcdefgh'))，mid函数只提供了第1个参数，第2个参数和第3个参数都使用默认值，等价于print(mid('abcdefgh',0,1))，返回结果为 'a'。
  - 第6行代码 print(mid('abcdefgh',2))，mid函数提供了第1个参数和第2个参数，第3个参数使用默认值，等价于print(mid('abcdefgh',2,1))，返回结果为 'c'。
  - 第7行代码 print(mid('abcdefgh',2,4))，mid函数提供了全部参数，返回结果为 'cdef'。

18.3.2 可选参数应用案例：模拟 vlookup 函数的应用

本案例根据“销售表”工作表中 A 列的产品名称，到“单价表”工作表中查询对应的产品价格，再用查询到的产品价格乘以产品数量得到产品金额，将产品金额写入“销售表”工作表的 C 列单元格，如图所示。

本案例的编程思路是自定义一个类似于 Excel 工作表函数 vlookup 的函数。



	A	B
1	产品	价格(元)
2	A	2.5
3	B	6
4	C	7
5	D	1.5
6	E	4

单价表

	A	B	C
1	产品	数量	金额(元)
2	B	12	
3	C	15	
4	B	6	
5	A	6	
6	D	8	
7	E	15	
8	A	6	
9	A	8	
10	C	12	
11	D	8	

销售表

处理前

	A	B	C
1	产品	数量	金额(元)
2	B	12	72
3	C	15	105
4	B	6	36
5	A	6	15
6	D	8	12
7	E	15	60
8	A	6	15
9	A	8	20
10	C	12	84
11	D	8	12

销售表

处理后

```
# 自定义vlookup函数
def vlookup(val,lst,num=1):
```

```
def vlookup(val,lst,num=1):
    r = lst[0].index(val) # 对lst列表第1个元素使用index函数进行查找。得到返回值赋值给r变量。
    v = lst[num][r] # 再使用num和r变量确定截取位置。
    return v # 将截取的值返回给函数
# 自定义函数应用
import xlrd # 导入读取xls文件的库。
from xlutils.copy import copy # 导入复制工作簿函数。
wb = xlrd.open_workbook('./30-functions-1.xls') # 读取工作簿。
ws_price = wb.sheet_by_name('单价表') # 读取工作簿下的工作表。
ws_Sale = wb.sheet_by_name('销售表') # 读取工作簿下的工作表。
nwb = copy(wb);nws=nwb.get_sheet('销售表') # 复制工作簿，以及读取副本工作簿下的工作表。
lst = [ws_price.col_values(0)[1:],ws_price.col_values(1)[1:]] # 读取'单价表'下的产品列和单价列。
for row_num in range(1,ws_Sale.nrows): # 循环'销售表'中的行号。
    val1 = ws_Sale.cell_value(row_num,0) # 读取'销售表'A列的品名。
    val2 = ws_Sale.cell_value(row_num,1) # 读取'销售表'B列的数量。
    val3 = vlookup(val1,lst)*val2 # 将val1中的品名去lst列表查找，返回对应的单价，再
```

```
乘以val2的数量，得到金额。
    nws.write(row_num,2,val3) # 将计算出来的金额写入'销售表'的C列单元格。
nwb.save('./30-functions-2.xls') # 保存副本工作簿。
```

- 第 2～5 行代码自定义查询函数 vlookup(val,lst,num=1)，该函数的参数说明如下。
    - val:必选参数,要查询的值。
    - lst:必选参数,要查询的列表,列表中的每个元素也是列表,查询必须在第0个元素的列表中。
    - num:可选参数,确认要返回列表中第几个元素的列表中的值,默认值是1。
- 第 17 行代码 val3＝vlookup(val1,lst)＊val2,
    - 其中 vlookup(vall,lst)中的 val1 表示要查询的产品,
    - lst 是放置“单价表”的两列数据 [['A','B','C','D','E'],[2.5,6.0,7.0,1.5,4.0]],
    - 第 3 个参数是使用的默认值1,也就是如果查询成功,则返回［2.5,6.0,7.0,1.5,4.0］中对应位置的价格。
    - 最后将查询到的价格乘以 val2 中的数量,得到金额后赋值给 val3 变量。

18.4 关键字参数的写法及应用

18.4 关键字参数的语法及应用

在函数调用时，通过“参数＝值”的形式传入实参的方式叫作关键字参数，可以让函数更加清晰、易于使用，同时参数的书写可以不按顺序。

18.4.1 关键字参数

关键字参数分为普通关键字参数和命名关键字参数。普通关键字参数可以用“参数＝值”的方式传入实参，也可以按参数位置的先后顺序直接传入实参。命名关键字参数必须用“参数＝值”的方式传入实参。

下面看看普通关键字参数与命名关键字参数的不同写法与传参方式，代码如下所 示。

```
# 自定义函数
def counter1(iterable,min,max):# 普通关键
```

```
字参数。
    lst=[v for v in iterable if v>=min
and v<=max]
    return len(lst)
def counter2(iterable,*,min,max):# 命名
关键字参数。
    lst=[v for v in iterable if v>=min
and v<=max]
    return len(lst)
# 调用自定义函数
print(counter1([2,3,8,3,4,5,9],3,8)) #
按位置传递实参。
print(counter1(max = 8,min = 3,iterable
= [2,3,8,3,4,5,9])) # 按关键字参数传递实
参。
print(counter1([2,3,8,3,4,5,9],3,max =
8)) # 部分按位置传递实参，部分按关键字参数传递
实参。
print(counter2([2,3,8,3,4,5,9],3,8)) #
按位置传递实参会出错。
print(counter2(max = 8,min = 3,iterable
= [2,3,8,3,4,5,9])) # 按关键字参数传递实
参。
print(counter2([2,3,8,3,4,5,9],max =
8,min = 3)) # 部分按位置传递实参，部分按关键
字参数传递实参。
```

- 第 2～7 行代码自定义条件计数函数 counter1(iterable,min,max)和counter2(iterable,＊,min,max),它们的参数都相同,参数说明如下。
    - iterable:必选参数,提供要判断的序列数字。

- min:必选参数,定义范围最小值。
- max:必选参数,定义范围最大值。

通过观察发现,counter1 函数和 counter2 函数基本一样,唯一不同点就是 counter2 函数的参数中有一个 `*` (星号),它起什么作用呢？ 星号前面的是普通的关键字参数,星号后面的是命名关键字参数。也就是说,星号起到分隔的作用,便于用户区分。接下来调用一下这两个函数,看它们的使用方法有何区别。

- 第 9 行代码 print(counter1([2,3,8,3,4,5,9],3,8)),

  普通关键字参数可以按位置传参数,只要顺序不乱,返回的结果就不会错。

- 第 10 行代码 print(counter1(max＝8,min＝3,iterable＝[2,3,8,3,4,5,9]),这是标准的关键参数传递方式,传递参数的顺序可以打乱,只要把形参和实参一一对应上就可以了。

- 第 11 行代码 print(counter1([2,3,8,3,4,5,9],3,max＝8)),这是位置参数与关键字参数混合传递方式,同时包含这两种形式,一定是先位置参数,后关键字参数。前两个参数是按照位置传递参数的,第3个参数是使用关键字参数传递参数的。

- 第 12 行代码 print(counter2([2,3,8,3,4,5,9],3,8)),因为 counter2 函数中有命名关键字参数,所以**这种位置传参方式会出错**。min 和 max 两个参数必

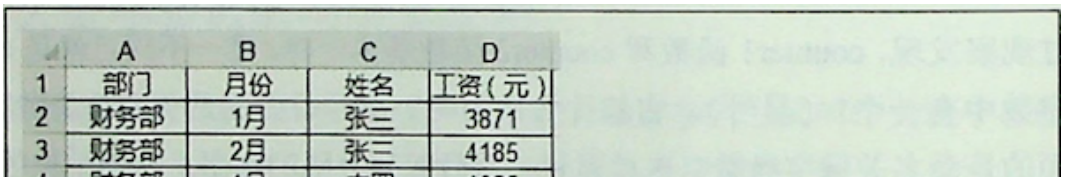
须按“参数＝值”的方式来书写,前面的iterable参数可以不按“参数＝值”的方式来书写。

- 第 13 行代码 print(counter2(max＝8,min＝3,iterable＝2,3,8,3,4,5,9])),counter2 函数的所有参数都按“参数＝值”的方式来书写,这种方式也能正常运行。

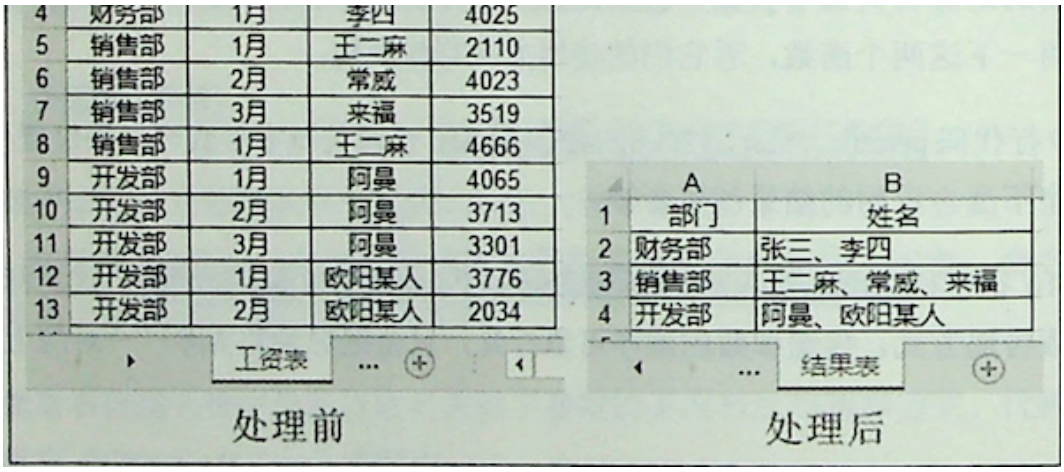
- 第14行代码 print(counter2([2,3,8,3,4,5,9],max＝8,min＝3)),counter2 函数按混合传参方式来书写也是可以的,注意位置传参方式要写在最前面。

18.4.2 关键字参数应用案例：分类合并字符串

本案例以“工资表”工作表的“部门”列为基准，合并“姓名”列，如图所示。本案例的编程思路是创建一个字典，将指定列的数据作为键，将指定的另一列的数据作为值，循环装入字典中，再对字典中每个键对应的值去重，然后根据指定的分隔符合并，最后将结果返回给自定义的函数即可，也就是把整个数据处理放在自定义函数中。



	A	B	C	D
1	部门	月份	姓名	工资（元）
2	财务部	1月	张三	3871
3	财务部	2月	张三	4185



4	财务部	1月	李四	4025
5	销售部	1月	王二麻	2110
6	销售部	2月	常威	4023
7	销售部	3月	来福	3519
8	销售部	1月	王二麻	4666
9	开发部	1月	阿曼	4065
10	开发部	2月	阿曼	3713
11	开发部	3月	阿曼	3301
12	开发部	1月	欧阳某人	3776
13	开发部	2月	欧阳某人	2034

处理前

	A	B
1	部门	姓名
2	财务部	张三、李四
3	销售部	王二麻、常威、来福
4	开发部	阿曼、欧阳某人

处理后

```
# 自定义combine函数。
def
combine(range,join_range,delimiter='
'):
    dic={} # 初始化dic变量为空字典。
    for x,y in zip(range,join_range): #
将range和join_range接收的值使用zip函数做组合
转换，并赋值给 x,y 变量。
        if not x in dic: # 如果x在dic字典
中不存在。
            dic[x]=[y] # 则以x为键，y为
值，创建键值，注意y是写在列表中的。
        else: # 否则。
            dic[x].append(y) # 如果x键存
在，则将y添加到对应的值列表中。
    l=[(k,delimiter.join([str(i) for i
in dict.fromkeys(i)])) for k,i in
dic.items()] # 将键对应值列表去重合并。
    return l # l变量是列表，列表中的元素是元
```

```
组类型，元组中分别存储是dic字典的键，及dic字典对应值合并后的字符串。
# 自定义函数应用
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导计工作簿复制函数。
wb=xlrd.open_workbook('./31-functions-1.xls');ws=wb.sheet_by_name('工资表') # 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('结果表') # 复制工作簿，以及读取副本工作簿下的工作表。
row_num=0 # 初始化row_num变量为。
for x,y in combine(delimiter='、',range=ws.col_values(0),join_range=ws.col_values(2)): # 调用combine函数。
    nws.write(row_num,0,x) # 将range参数中的值去重后写入'结果表'工作表A列。
    nws.write(row_num,1,y) # 将join_range参数中的值去重合并再写入'结果表'工作表B列。
    row_num +=1 # 累加row_num变量。
nwb.save('./31-functions-2.xls') # 保存副本工作簿。
```

- 第 2～10 行代码,自定义合并函数 combine(range,join_range,delimiter=''),该函数的参数说明如下。
    - range:必选参数,指定基准列。
    - join_range:必选参数,指定合并列。
    - delimiter:可选参数,对列进行合并时指定的分

隔符,默认值是空格。

- 第 17 行代码 for x,y in combine(delimiter='、',range＝ws.col*values(0),join*range＝ws.col_values(2))：，调用了combine 函数。

    - delimiter 参数指定的分隔符是'、'；
    - range 参数指定的基准列是ws.col_values(0),也就是“工资表”的A列；
    - join*range 参数指定的合并列是 ws.col*values(2),也就是“工资表”的B列。
    - combine函数的3个参数顺序因为使用的是关键字参数方式,所以顺序可以打乱。
    - combine 函数的返回值是列表,列表中的元素是元组,元组中包含了部门和对应的姓名,结果为［('部门','姓名'),('财务部','张三、李四'),('销售部','王二麻、常威、来福'),('开发部','阿曼、欧阳某人')］。
    - 最后将列表中的值循环写入“结果表”工作表即可。

18.5 不定长参数的写法及应用

不定长参数是指定义的函数参数个数不确定，可以将不定长参数收纳在指定的容器中。

不定长参数分为以下两种。

- `*args`：以一个星号开头，然后接 arg 变量，这

- `*args`:以一个星号开头，然后接 arg 变量，这种参数接收的值存储在元组中，即 args 是一个元组类型。

- `**kwargs`:以两个星号开头，然后接 kwargs 变量，这种参数接收的值存储在字典中，即 kwargs 是一个字典类型。

args 与 kwargs 变量名称不是固定的，只是约定俗成的一种命名方式，用户可以修改成其他名称。不定长参数必须放在必选参数之后。

`*args` 和 `**kwargs` 接收的数据类型不限，完全根据需求而定。

18.5.1 不定长参数1

下面是一个小案例，代码如下所示。

```
# 自定义函数
def subtotal(iterable,*args):
    print(args) # 在屏幕打印args中的内容。
    return [fun(iterable) for fun in
args] # 根据args接收的函数返回处理结果。
# 调用自定义函数
print(subtotal([7,8,9,10],max,min,sum))
```

- 第 2～4 行代码自定义汇总函数 subtotal(iterable,*args)，该函数的参数说明如下。
    - iterable:必选参数，提供要判断的序列数

    - iterable:必选参数，提供要判断的序列数字。
    - *args:可选参数，指定汇总函数。

- 第 6 行代码
  print(subtotal([7,8,9,10],max,min,sum))，
    - iterable变量的值是 [7,8,9,10] 列表，
    - 汇总函数是max、min、sum，这 3 个函数都将存储在args变量中。
    - 运行第 3 行代码 print(args)，屏幕上的输出结果为（＜built-in function max＞，＜built-infunction min＞，＜built-in function sum＞），可以看到元组的每个元素是函数对象。
    - 用每个函数去处理 [7,8,9,10] 即可，此行代码只是为了便于理解而写的，可以省略不写。
    - 最后调用 subtotal 函数，返回的结果为 [10,7,34]。

18.5.2 不定长参数2

下面是一个小案例，代码如下所示。

```
# 自定义函数
def subtotals(iterable,**kwargs):
    print(kwargs) #在屏幕打印args中的内容。
```

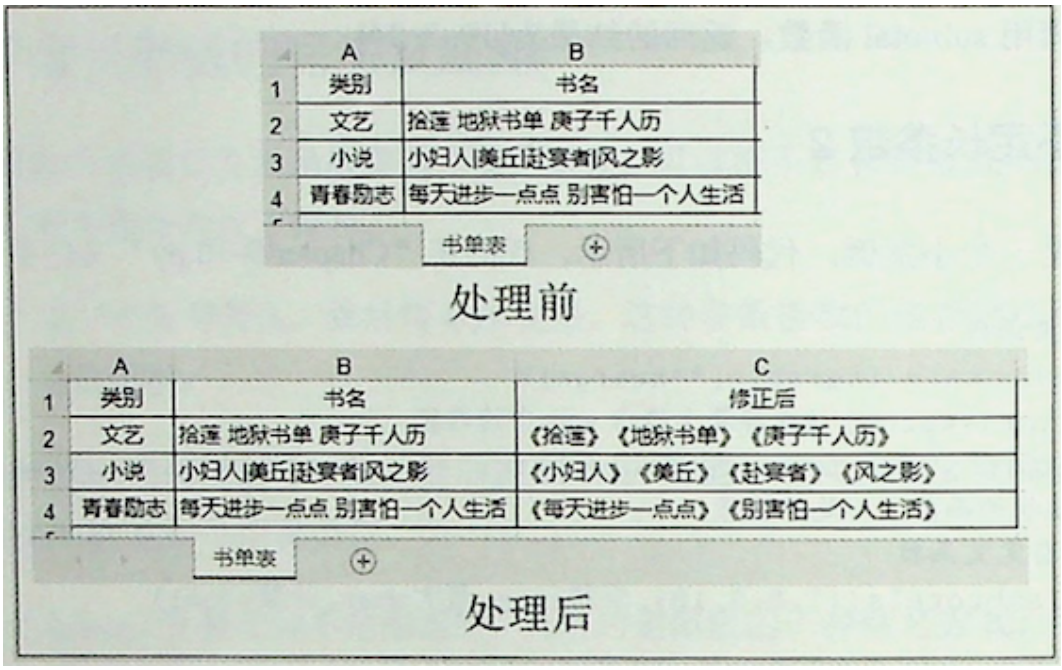
```
    return [(key,item(iterable)) for
key,item in kwargs.items()] # 根据args接
收的函数返回处理结果。
# 调用自定义函数
print(subtotals([7,8,9,10],求和=sum,最大
=max,计数=len))
```

- 第2～4行代码自定义汇总函数 subtotals(iterable,**kwargs)。
    - iterable:必选参数,提供要判断的序列数字。
    - **kwargs:可选参数,按字典的键值方式指定汇总函数。
- 第6行代码 print(subtotals([7,8,9,10],求和＝sum,最大＝max,计数＝len)),
    - iterable变量的值是[7,8,9,10]列表,
    - 汇总函数的指定方式是求和＝sum,最大＝max,计数＝len,这3对键值都将存储在 kwargs 变量中。
- 运行第3行代码print(kwargs),屏幕上的输出结果为{ ' 求和 ' ： ＜built-in function sum＞, ' 最大 ' ： ＜built-in function max＞, ' 计数： ＜built-in function len＞},可以看到字典中的键是汇总名称,值是对应的函数名称,用值的每个函数去处理 [7,8,9,10] 即可。此行代码是为了便于理解而写的,可以省略不写。
- 最后调用 subtotals 函数,返回的结果为[('求

和',34),('最大',10),('计数',4)]。

18.5.3 不定长参数应用案例：替换函数增强版

在本案例中，“书单表”工作表的 B 列是书名，书名之间使用空格 (' ')和竖线 ('|') 进行分隔，现在要用规范的书名号来分隔，修改前后的效果如图所示。



本案例的编程思路是定义一个可以替换指定多个旧字符串为统一新字符串的函数，然后调用该函数处理 B 列的书名即可。

```
# 自定义replaces函数。
def replaces(string,new,*old):
    for o in old: # 循环old参数中的元素赋值给o变量。
```

```
        string = string.replace(o,new)
# 将o变量中的值，替换为new变量中的值，最后再赋值给string参数。
    return string # 循环替换完成后，将string中的值返回给replaces函数。
# 自定义函数应用
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导计工作簿复制函数。
wb=xlrd.open_workbook('./32-books-1.xls');ws=wb.sheet_by_name('书单表') # 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('书单表') # 复制工作簿，以及读取副本工作簿下的工作表。
for row_num in range(1,ws.nrows): # 循环'书单表'的行号，赋值给row_num。
    val1=ws.cell_value(row_num,1) # 获取B列单元格的书名，并赋值给val变量。
    val2='《'+replaces(val1,'》《','','|')+'》' #将' '和'|'替换为'》、《'，再补齐两端书名号，赋值给 val2 变量。
    nws.write(row_num,2,val2) # 将处理好后的val2变量写入C列单元格。
nwb.save(./32-books-2.xls') # 保存副本工作簿。
```

- 第 2～5 行代码，自定义替换函数

  replaces(string,new,*old)，参数说明如下。
  - string:必选参数，原字符串。
  - new:必选参数，指定要替换的新字符串。
  - *old:可选参数，指定替换的旧字符串，可以

old:可选参数，指定替换的旧字符串，可以指定多个。

- 第13行代码 val2='《'+replaces(val1,'》《','','|')+'》'，调用 replaces 函数，string 参数是 vall 变量，也就是 B 列单元格的书名。new 参数是“》《”，表示将要替换的新字符串，old参数获取的是（，T），表示将空格（ ' ' ）和竖线（ " ）都替换为“》《”。
- 最后将替换结果赋值给val2变量。

18.6 匿名函数的写法及应用

匿名函数指无须定义标识符的函数或子程序。

Python 中用 lambda 关键字定义匿名函数，只用表达式而无须声明。

使用匿名函数有一个好处，就是匿名函数没有名字，不必担心函数名冲突。此外，匿名函数也是一个函数对象，可以把匿名函数赋值给一个变量，这样变量名就相当于函数名，可以利用变量来调用该函数。

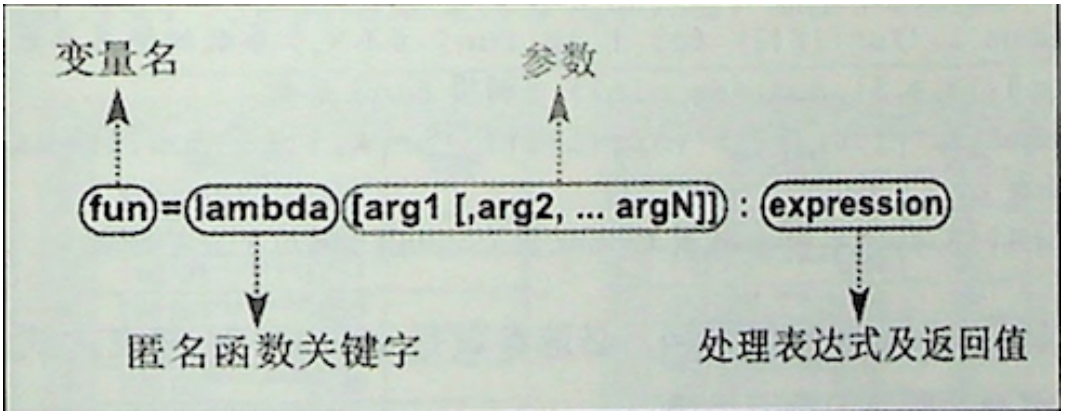
18.6.1 匿名函数的语法结构

在 Python 中，lambda 是一个关键字，用来引入表达式的语法。相比 def 函数，lambda 是单一的表达式，而不是语句块。

在 lambda 中只能封装有限的业务逻辑，lambda定义的匿名函数纯粹是为了编写简单函数而设计的，def 函

的匿名函数就样是为了编写简单函数而设计的，def 函数则专注于处理更大的业务。

匿名函数的语法结构如图所示。



下面看看匿名函数的定义和调用，代码如下所示。

```
print(lambda x:x*10) # 定义匿名函数。
print((lambda x:x*10)(20)) # 定义及调用匿名函数。
fun=lambda x:x*10 # 匿名函数赋值给变量。
print(fun(20)) # 将变量名当做函数名调。
```

18.6.2 匿名函数的常见书写方式

匿名函数与 def 定义的普通函数一样，可以有不同类型的参数，案例代码如下所示。

```
fun1=lambda x,y:x+y # 必选参数的匿名函数。
print(fun1(10,20)) # 调用fun1函数。
fun2=lambda x,y=20:x+y # 可选参数的匿名函数。
print(fun2(10)) # 调用fun2函数。
fun3=lambda l,*fun:[f(l) for f in fun]
# 不定长参数的匿名函数1
```

```
# 不定长参数的匿名函数1。
print(fun3([3,4,5],sum,max,min)) # 调用fun3函数。
fun4=lambda l,**fun:[k+':'+str(i(l)) for k,i in fun.items()] # 不定长参数的匿名函数2。
print(fun4([3,4,5],求和=sum,最大=max,最小=min)) # 调用fun4函数。
```

- 第 1 行代码 fun1 = lambda x,y:x＋y，必选参数是 x 和 y，处理表达式是将两个参数相加，再将匿名函数赋值给 fun1变量。

- 第 2 行代码 print(funl(10,20))，给 fun1变量传入参数并输出，结果为 30。

- 第 3 行代码 fun2=lambda x,y＝20:x＋y，x 是必选参数，y 是可选参数，默认值是 20，处理表达式是将两个参数相加，再将匿名函数赋值给fun2变量。

- 第 4 行代码 print(fun2(10))，x 参数接收的是 10，而y参数使用默认值 20，最后 fun2 的输出结果为 20.

- 第 5 行代码 fun3＝lambda l,*fun:[f(l) for f in fun],l*

  是必选参数,fun 是不定长参数,用元组存储数据,处理表达式是用 fun 元组接收到的函数去处理 l 参数中的值,再将匿名函数赋值给 fun3 变量。

  第 6 行代码 print(fun3([3,4,5],sum,max,min)) l 参

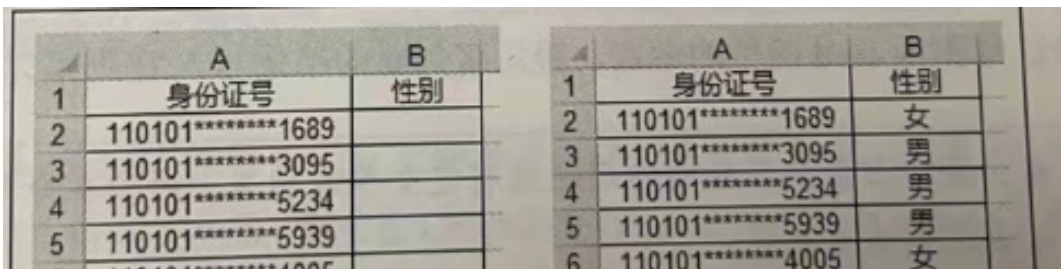
- 第 6 行代码 print(fun3([3,4,5],sum,max,min)),l 参数接收的是 [3,4,5],而 fun 参数接收的是(＜built-in function sum＞,＜built-in function max＞,＜built-in function min＞),最后 fun3 的输出结果为 [12,5,3]。

- 第 7 行代码 fun4＝lambda l,**fun:[k＋ ' : ' ＋ str(i(l))for k,i in fun.items()],l 是必选参数,**fun是不定长参数,用字典存储数据。处理表达式是用 fun 字典接收到的函数去处理1参数中的值,再将匿名函数赋值给 fun4 变量。

- 第 8 行代码 print(fun4([3,4,5],求和＝sum,最大＝max,最小＝min)),l 参数接收的是 [3,4,5],而fun参数接收的是 ｛ ' 求和：＜built-in function sum＞, ' 最大：＜built-in function max＞, ' 最小 ' ：＜built-in function min＞｝,最后fun4的输出结果为 [' 求和：12, ' 最大：5 ' , ' 最 小：3 ']。

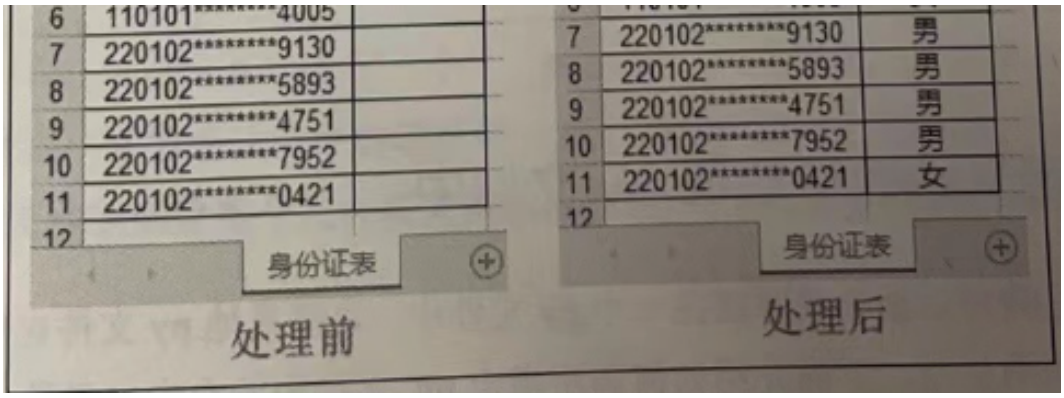
18.6.3 匿名函数应用案例：根据身份证号判断性别

本案例根据“身份证表”工作表A列单元格的身份证号第 17 位数字判断性别，奇数为男，偶数为女，然后将结果写入 B 列单元格，如图所示。



	A	B
1	身份证号	性别
2	110101********1689	
3	110101********3095	
4	110101********5234	
5	110101********5939	

	A	B
1	身份证号	性别
2	110101********1689	女
3	110101********3095	男
4	110101********5234	男
5	110101********5939	男
6	110101********4005	女



本案例的编程思路是使用匿名函数定义一个判断性别的函数，然后调用即可。

```
import xlrd # 导入读取xls文件的库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb=xlrd.open_workbook('./33-ids-1.xls');ws=wb.sheet_by_name('身份证表') # 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('身份证表') # 复制工作簿，并且读取副本工作簿中的工作表。
for row_num in range(1,ws.nrows): # 循环'身份证表'工作表的行号。
    card=lambda id:'男' if int(id[-2])%2==1 else '女' # 自定义根据身份证号判断性别的匿名函数。
    val=ws.cell_value(row_num, 0) # 获取A列的身份证号，并赋值给val变量。
    nws.write(row_num,1,card(val)) # 将val做为匿名函数card的参数，将判断出的性别写入B列单元格。
nwb.save('./33-ids-2.xls') # 保存副本工作簿。
```

18.7 自定义函数存放在 .py 文件中

自定义函数可以被单独存放在一个 **.py** 文件中，这样其他 **.py** 文件也可以调用，实现自定义函数的复用。

下面介绍如何调用指定 **.py** 文件中的自定义函数。我们将之前学习的一部分自定义函数放在 **fun.py** 文件中，然后在另外的 **.py** 文件中调用，代码如下所示。

```
# 平均函数
def average(lst):
    num=sum(lst)/len(lst) # 平均处理。
    avg=float('{:.2f}'.format(num)) # 格式化平均值。
```

```
    return avg # 返回平均值。

# 分段函数
def intercept(s,num,delimiter):
    s1=str(s) # 将要分段的对象转换为字符串类型。
    lst=[s1[n:n+num] for n in range(0,len(s1),num)] # 对数据进行分段处理。
    s2=delimiter.join(lst) # 合并分段的列表。
    return s2 # 将合并结果返回函数。

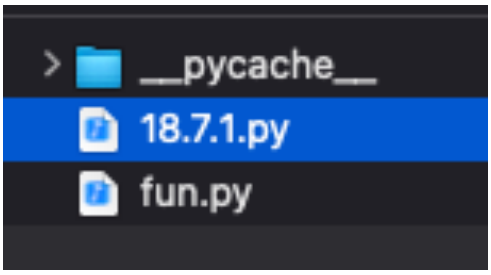
# 模仿vlookup
def vlookup(val,lst,num=1):
    r=lst[0].index(val) # 对lst列表第1个元素使用index函数进行查找。得到返回值赋值给r变量。
    v=lst[num][r] # 再使用num和r变量确定截取位置。
    return v # 将截取的值返回给函数

# 分类合并字符串
def combine(range,join_range,delimiter=' '):
    dic={} # 初始化dic变量为空字典。
    for x,y in zip(range,join_range): # 将range和join_range接收的值使用zip函数做组合转换，并赋值给x,y变量。
        if not x in dic: # 如果x在dic字典中不存在。
            dic[x]=[y] # 则以x为键，y为
```

```
                dic[x]=[y] # 则以x为键，y为
值，创建键值，注意y是写在列表中的。
        else: # 否则。
            dic[x].append(y) # 如果x键存
在，则将y添加到对应的值列表中。
        l=[(k,delimiter.join([str(i)
for i in dict.fromkeys(i)])) for k,i in
dic.items()] # 将键对应值列表去重合并。
    return l # l变量是列表，列表中的元素是元
组类型，元组中分别存储是dic字典的键，及dic字典
对应值合并后的字符串。
```

18.7.1 函数定义在单独 .py 文件中

18-7-1.py 文件与 **fun.py** 文件存放在同一个文件夹中，如图所示。



fun.py 文件中存放的是自定义函数，要求在 **18-7-1.py** 文件中调用 **fun.py** 文件中的函数。

顺便说一句，在图中还有一个“**pycache**”文件夹，是在导入 **fun.py** 文件时产生的文件夹，目的是让程序启动更快一点，当用户的脚本更改时，它将被重新编译，如果删除文件或删除整个文件夹并再次运行程序，它将重新出现，一般忽略即可。

```
import fun #导入函数文件名。
```

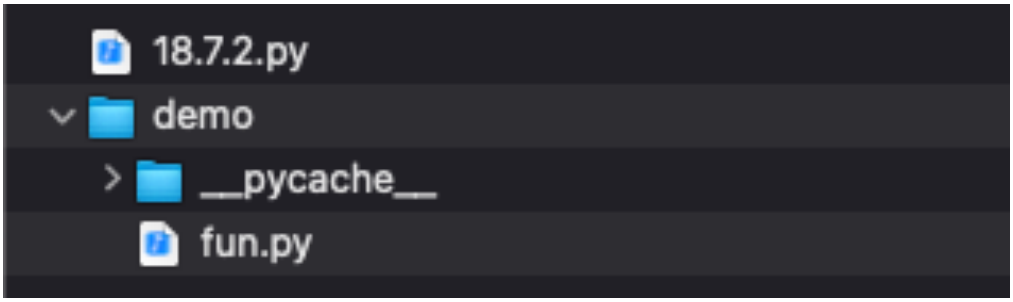
```
print(fun.average([1,2,3,4,5,6])) #调用fun文件下的average函数。
print(fun.intercept('5120124575',4,'-')) #调用fun文件下的intercept函数。

from fun import * #导入函数文件名下的所有函数。
print(intercept('4523465745',4,'、')) #直接调用函数

from fun import average #导入函数文件名下指定的函数。
print(average([4,2,7])) #直接调用函数
```

- 第 1 行代码 import fun，直接导入函数文件名，注意不要写扩展名 `.py` 。
- 第 2 行代码和第 3 行代码调用函数，fun.average 表示调用 fun 文件中的 average 函数。同理，fun.Intercept表示调用 fun 文件中的 intercept 函数。
- 第 5 行代码 from fun import * ，表示导入 fun 件中的所有函数。
- 第 6 行代码直接调用 intercept 函数。
- 第 8 行代码 from fun import average，表示只导入 fun 文件中的 average 函数。
- 第 9 行代码就可以调用 average 函数了，但也只能调用 average 函数，因为只导入了 average 函数。

18.7.2 函数定义在文件夹中



调用不同文件夹中的函数的案例代码如下。

```
from demo.fun import * # 导入文件夹下指定的fun.py文件下的所有函数。
print(average([1,2,3,4,5,6])) # 直接调用函数
print(intercept('5120124575',4,'-')) # 直接调用函数

from demo.fun import average # 导入文件夹下指定的fun.py文件下的所有函数。
print(average([43,52,12])) # 直接调用函数
```

- 第 1 行代码 from demo.fun import *，其中 demo.fun 表示 demo 文件夹中的 fun.py 文件，导入该文件中的所有函数。
- 第 2 行代码和第 3 行代码直接调用 fun.py 文件中的 average 函数和 intercept 函数。
- 第 5 行代码 from demo.fun import average，表示导入 demo 文件夹中 fun.py 文件的 average 函数。
- 第 6 行代码直接调用average函数，但也只能调用average函数，因为只导入了该函数。

在 Python 中，**包**和**库**是两个常见的概念，它们经常用来组织和管理代码。以下是它们的定义和操作方式：

---

19. Python 的包和库

19.1 Python 库的定义

- 包是一个 **Python 模块的集合**，用来组织 Python 模块（文件）结构化。
- 包的本质是一个带有 `__init__.py` 文件的目录。
- 通过包，你可以创建模块的层次结构，用于更清晰地组织和复用代码。

19.2 库的示例结构

```
my_package/
    __init__.py
    module1.py
    module2.py
    subpackage/
        __init__.py
        module3.py
```

`__init__.py` 文件：

- 该文件可以为空，但它标志着该目录是一个包。

- 可以在其中定义包的初始化逻辑（例如导入模块、定义包的变量等）。

19.3 Python 中的库定义

- 库是一个更宽泛的概念，指一组有用的功能或工具的集合，通常由一个或多个模块或包组成。
- 在 Python 中，库可以是标准库（内置库，如 `os`、`math`）、第三方库（如 `NumPy`、`Pandas`）或自定义库。

19.4 包与库的关系

- *包*是用来组织代码的工具，是库的一部分。
- 一个库可以由多个包组成。

19.5 安装第三方库

- 使用 `pip` 安装库。

  ```
  pip install 库名
  ```

  例如，安装 `requests`：

  ```
  pip install requests
  ```

- 查看已安装的库：

```
pip list
```

- 升级库：

```
pip install --upgrade 库名
```

19.6 导入包或库

- 导入整个模块：

```
import os  # 导入标准库 os
import my_package.module1  # 导入自定义包的模块
```

- 使用别名简化调用：

```
import numpy as np
import pandas as pd
```

- 导入特定功能：

```
from math import sqrt  # 从 math 库导入 sqrt 函数
from my_package.module1 import my_function  # 从模块中导入函数
```

19.7 创建自定义包

1. 创建包的目录结构：

```
my_package/
    __init__.py
    module1.py
    module2.py
```

2. 编写代码：
    - `module1.py`：

      ```
      def hello():
        return "Hello from module1"
      ```

    - `module2.py`：

      ```
      def add(a, b):
        return a + b
      ```

3. 使用自定义包：
    - 添加到系统路径：

      ```
      import sys
      sys.path.append('/path/to/my_pac
      kage')
      ```

    - 导入模块并使用：

      ```
      from my_package.module1 import
      ```

```
hello
print(hello())  # 输出: Hello
from module1
```

19.8 安装本地库（开发阶段使用）

- 使用 `setup.py` 文件定义库，并安装：

```
my_project/
  my_package/
      __init__.py
      module1.py
  setup.py
```

- `setup.py` 示例：

```
from setuptools import setup,
find_packages

setup(
  name="my_package",
  version="0.1",
```

```
    packages=find_packages(),
)
```

- 安装到本地开发环境：

```
pip install -e .
```

19.9 常见库操作

- **查找库的路径**：

```
import numpy
print(numpy.__file__)
```

- **卸载库**：

```
pip uninstall 库名
```

- **查看库文档**：

```
help(模块名)
```

总结来说，**包是代码的组织单位**，**库是功能的集合**，它们在 Python 中相辅相成。通过合理地组织包和使用库，你可以构建更清晰、复用性更高的项目。